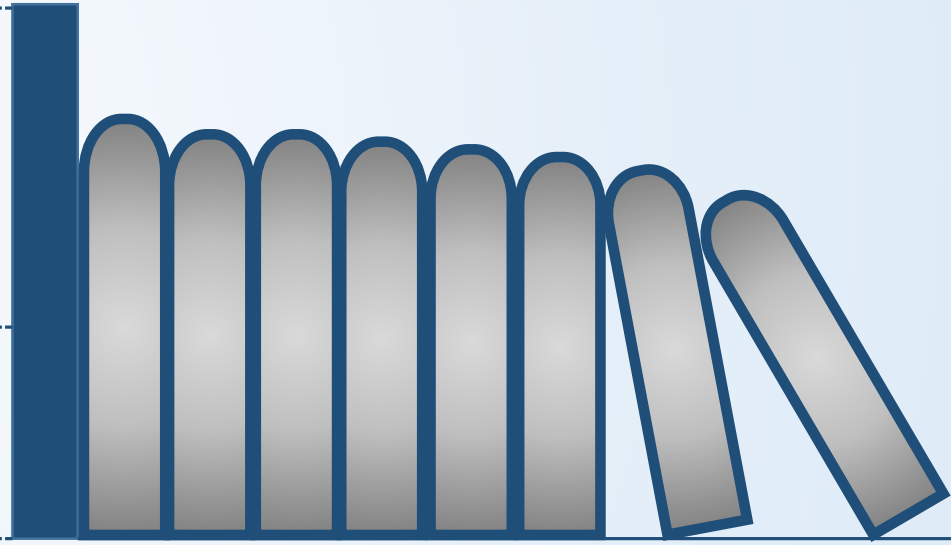
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

RSA 共享素因子攻击



2026年10月13日星期二



实验目的

- 掌握 RSA 私钥生成的完整数学链条
- 理解RSA 素因子攻击的原理
- 实现单对模数检测与私钥恢复
- 建立 “密钥生成缺陷 → 批量破解” 的因果认知



本次实验的背景

- 某单位有 1000 台自研加密设备，出厂时各自生成 RSA 密钥对
- 公钥登记在册 —— 也就是说，`n` 和 `e` 是**公开**的
  - 清单就是 `data/moduli.txt`：1000 行十六进制，每行约 256 字符
  - 位长有 1023 和 1024 两种 —— **这是正常的**，别为它写校验
- 运维方还交来一份**加密的通知**，会话密钥被其中一台的公钥加密了
  - 而那台设备的私钥「随设备下线丢失了」

我们的任务：**把它们的私钥恢复出来**



实验内容

四个任务，从**数学地基 → 原理认知 → 分析能力，** **最终得到私钥解开加密文件。**

任务	内容
任务一	数学基础（扩展欧几里得 + 模逆）
任务二	扫描 1000 个模数（拆开其中的一对）
任务三	分解 1000 个模数（扫描+分类+恢复全部私钥）
任务四	解开 PBP 文件



实验原理

- 加密算法

$$c \equiv m^e \bmod n$$

- 解密算法

$$m \equiv c^d \bmod n$$

Tips: 要求m<n，如果m>n，需要进行分组。

加密时首先将明文比特串分组，使得每个分组对应的十进制数小于n ，即分组长度小于 $\log_2 n$ 。



实验原理-RSA 算法的三个数

符号	是什么	怎么来的	公开?
n	模数	两个随机素数相乘：$n = p \cdot q$	**公开**
e	公开指数	随便挑一个与 φ(n) 互素的数（常用 65537）	**公开**
d	私钥指数	**算出来的**：$d = e^{-1} \bmod \phi(n)$	**保密**

算法推导链：

– 欧拉函数 φ(pq) = (p − 1)(q − 1)

– 欧拉定理　a^φ(n) ≡ 1 (mod n)　(当 gcd(a, n) = 1)

– 取 e · d ≡ 1 (mod φ(n))，于是 (m^e)^d ≡ m (mod n)

d 之所以能当私钥，全是因为它和 e 在模 φ(n) 下互为逆元

> 重点在最后一行：
> **攻击者的目标不是「分解 n」，是「拿到 φ(n)」。**



实验原理—RSA示例

RSA-密钥

- 两个素数: p=17, q=11
- 计算n=pq=17*11=187
- 计算$\varphi(n)$=(p-1)(q-1)=16*10=160
- 选择e,其中gcd(e,160)=1,假设e=7
- 求解d, 其中ed=1 mod 160,2<d<160

  d=23, 验证 23*7=161=1*160+1

公钥PU e=7 私钥PR d=23 n = 187

RSA-加密/解密

- M=88 (88<187)
- 加密

  $C=88^7 \bmod 187 = 11$
- 解密

  $M=11^{23} \bmod 187 = 88$

把这三个数放大到 1024 位 —— 就是你在本实验里要处理的东西。
数学一个字没变，只是数字长了。



实验原理 –从 n 到 φ(n)：三条路

路线一：分解 n —— 对 1024 位的合数走不通，算力超出宇宙年龄

路线二：猜测 φ(n) —— 不可行：φ(n) 和 n 一样大，穷举没有意义

路线三：从别人的模数里借一个素数 —— 本实验的攻击路线

– 两台设备共享一个素数：$\gcd(n_1, n_2) = p$，于是 $q_1 = n_1/p$，$d_1 = e^{-1} \bmod \phi(n_1)$

– **全程没有任何「分解」发生**

$n_1 = p \cdot q_1$　　$n_2 = p \cdot q_2$　　← 两台设备用了同一个素数 p

$\gcd(n_1, n_2) = p$　　← 一次 gcd，代价微秒级

$q_1 = n_1 // p$　　← 剩下的全是算术

$d_1 = \text{modinv}(65537, (p-1) * (q_1-1))$　　← 私钥到手

一次 gcd 约 12 微秒；分解一个 1024 位的合数，远超宇宙年龄。

差的不是算法优劣 —— 是「两台设备用了同一个素数」这一个事实。



实验原理 – 为什么会有相同的素因子?

素数不是「想」出来的，是**抽**出来的：512 位附近平均抽 **355 次**才中一个素数

– 每次迭代都依赖随机数；问题出在开机后的头几秒 —— **熵池还没攒够**

– 而 `/dev/urandom` 熵不足时**不阻塞**：照样给字节，只是更容易预测

出了什么事	留下的痕迹	用什么检测	能拿到私钥?
两台设备抽到**同一个素数**	两模数有非平凡公因子	两两 `gcd`	**能**
一台设备**把同一个素数用了两次**	`n` 是完全平方	`isqrt`	**能**
两台设备的素数落在同一个**窄区间**	`p`、`q` 挨得很近	Fermat 分解	**能**
设备出厂时**克隆了同一份密钥**	模数完全相同	排序去重	**不能**

> 除了第一种情况，下面其他三种情况也存在被攻击的可能性。



实验步骤

task1_math.py	10
task2_pair.py	6
task3_scan.py	9
task4_decrypt.py	4

- **空长这样：`FILL[1]` 或单独一行的 `FILL[2]`**
  - 每个 `TODO` 都标了编号，和指导书附录 E 的清单一一对应
- **把 `FILL[n]` 这几个字符删掉，在原位写代码**
  - `FILL[3] = math.isqrt(n)` ✗ 错！会把 `FILL` 这个名字绑到别的东西上
  - `r = math.isqrt(n)` ✓ 对！把 `FILL[3]` 删掉换上这个
- **一处填空 = 一个完整表达式或一条完整语句**
- **填空只在函数体里**，所以任何文件都可以 `--help`、可以 import



实验步骤

任务一　数学地基

```
$ python task1_math.py --selftest
```

`FILL[1]~FILL[4]`　`egcd` 四行：循环前的两个初值 + 循环里两行「同时更新」

`FILL[5]~FILL[7]`　`modinv` 三行：调 egcd、不互素抛错、归约到 `[0, m)`

`FILL[8]~FILL[10]`　`totient` 三行：**`p != q` 的分支** + 两种情形

坑	症状	唯一防线
元组赋值拆成两句	不报错，答案**全错**	Bézout 恒等式检查
忘了把 d 归约到 `[0, m)`	不报错，d 是**负数**	**范围检查** —— 前三道都抓不住
φ(p²) 写成 `(p-1)²`	不报错，最后**解不开**	自检里的 φ(7,7) 预言机



实验步骤

任务 二：拆开一台设备

```
$ python task2_pair.py        # 本机约 6~7 秒
```

`FILL[1]` 起点 `start = i + 1`：无序对只算一次，**不和自己配**

`FILL[2]` `math.gcd(moduli[i], moduli[j])` —— **别手写欧几里得**，实测快一个数量级

`FILL[3]`、`FILL[4]` `if g > 1`：命中**全部先收集**，形状是 `(i, j, g)`

`FILL[5]` 把三种 `g` 的情形分开 —— **一整块**；`FILL[6]` `modinv(e, phi)`

情形	含义	怎么办
`g == 1`	两台设备无关	不是这一对，看下一对
`1 < g < n`	**共享素因子**	`p = g`，接着算 φ 和 d
`g == n`	两个模数**完全相同**	异常，但**拆不开**



实验步骤

任务二的输出

```
命中       设备 i    设备 j    gcd 的位长
──────────────────────────────────────────────
        […]   […]   512 位
        […]   […]   512 位
        […]   […]   512 位
        […]   […]   512 位
        […]   […]   1024 位     ← 和模数一样长!
```

512 位 = 模数的**一半** —— 符合「两个 512 位素数之积」的预期

1024 位 = 模数的**全部** —— 说明 gcd 把整个模数吐了出来

– 也就是两个模数**完全相同**：`g == n`

– 这是**异常**，但攻击者从里面**什么也拿不到** —— 他知道 n



实验步骤

任务三：扫描 + 分类 + 恢复全部私钥

```
$ python task3_scan.py
$ python verify.py              # 性质检查
$ python verify.py --appendix-b # 再加一层：与附录 B 核对
```

`FILL[1]~FILL[3]` 分类：**`1 < g < moduli[i]` 才是「共享」；`g == n` 是「重复」**

`FILL[4]、FILL[5]` 完全平方：`math.isqrt` + 判定块（两两 gcd 对它**完全沉默**）

`FILL[6]~FILL[9]` 恢复：借 `p` → `q = n // p` → `d = modinv(PUBLIC_E, phi)` → `phi = totient(p, q)`

> ⚠ $\phi(p^2) = p(p-1)$，**不是** $(p-1)^2$。
> 两者差一个 512 位的数。算错了**不报错**，只是最后解不开。
> 别手写公式 —— 用你任务一自己写的 `totient`。



实验步骤—为什么三个数不一样

命中条数 ≠ 命中台数 ≠ 可分解台数

```
★ 三个数，别混
================================================================
命中条数      5 条
命中涉及设备  7 台    [217, 358, 583, 604, 762, 839, 921]
真正可分解    6 台    [217, 358, 471, 604, 839, 921]
```

数	是什么	为什么和别的不同
命中条数	有几**对**模数共享素因子	3 台设备互相共享时，它们之间有 **3 对**
命中涉及台数	有几**台**设备被牵涉	完全平方那台**一条命中都没有**，所以不在里面
真正可分解台数	攻击者能拿到私钥的台数	模数重复那几台有命中，但**拆不开**

3 台设备共享一个素数 p：
(7, 8, p) (7, 9, p) (8, 9, p) ← 3 条命中，只涉及 3 台设备

对比：命中条数 > 命中台数 ← 第一个分叉
可分解台数 ≠ 命中台数 ← 第二个分叉



实验步骤—另外的一类问题$n = p^2$

$n = p \times p$ —— 两个「素因子」是**同一个数**

所以它和**其他每一个模数**的 gcd 都是 1

– 别的模数都是独立随机的，没有一台持有这个 p

而「自己和自己」不算一对 —— 扫描跑完，它**一条命中都没有**

r = isqrt(n) → 若 r * r == n，那 r 就是它的素因子（两个都等于 r）

做法	1,000 个模数的耗时
两两扫描（499,500 次 gcd）	约 6 秒
开方检查（1,000 次 isqrt）	**约 3 毫秒** ← 快 **约 2000 倍**

> **发现它靠的是「知道还有这一类」，不是靠算力。**



实验步骤

任务四：　解开那份文件

`FILL[1]~FILL[4]`　私钥运算 → **定长** `i2osp` → 用给好的解码器 → 反向确认

PBP 是一个三层容器：**RSA 密文 → IV → AES 密文体**

- RSA 加密的是 PKCS#1 v1.5 块：`EM = 0x00 || 0x02 || 随机填充 || 0x00 || session_key`
- **RSA 作用在这一整块上**，不是作用在里面的会话密钥上

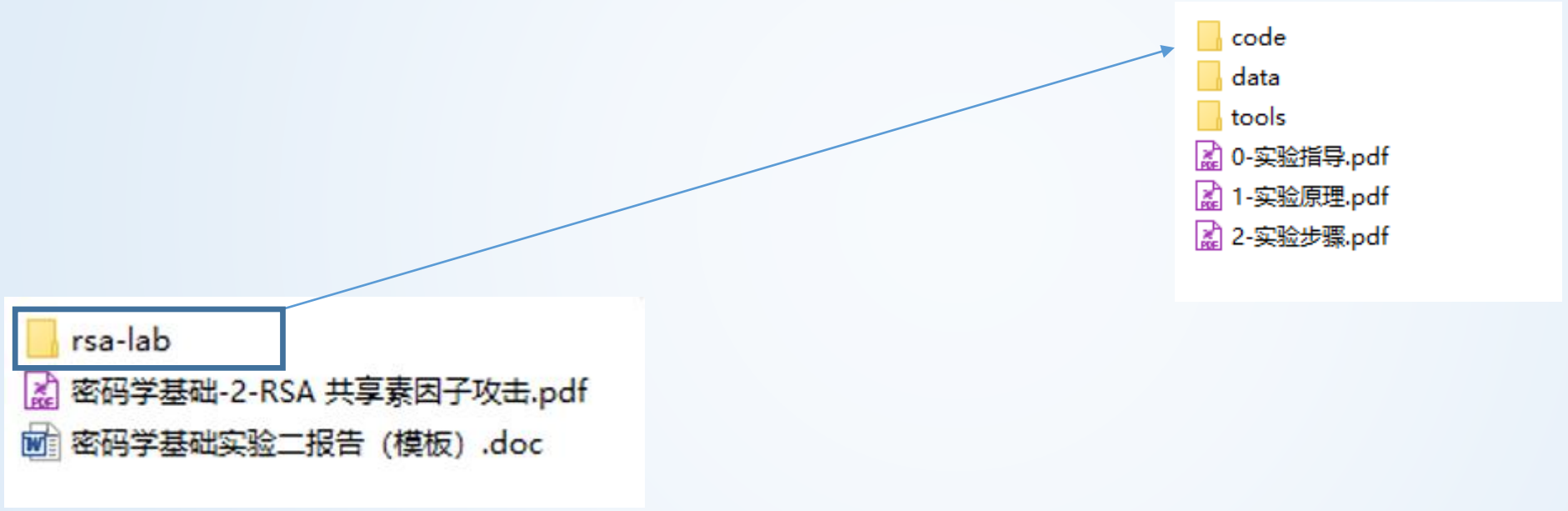
为什么包这么多层？因为 RSA **直接加密大文件不行** —— 慢，且一次只能加密一个块

- 这是**混合加密**：RSA 保护会话密钥，会话密钥保护数据（TLS、PGP、SSH 都用这个结构）

1. EM = c^d mod n　　← RSA 私钥运算
2. 剥掉 PKCS#1 填充 → session_key
3. 明文 = AES^{-1}(session_key, IV, 密文体)
4. 去掉 PKCS#7 填充



实验材料


code
data
tools
0-实验指导.pdf
1-实验原理.pdf
2-实验步骤.pdf
rsa-lab
密码学基础-2-RSA 共享素因子攻击.pdf
密码学基础实验二报告（模板）.doc



实验要求

- 提交内容

#	交什么
1	`data/hit.json`
2	`data/weak_devices.json`
3	`data/homework.pdf`
4	`code/task1-4`
3	实验报告

- 截止时间

下一次实验课前提交至HITsz Grader 作业提交平台，具体截止日期参考平台发布。

- 登录网址：： https://grader.hitsz.edu.cn/
- 推荐浏览器： Chrome

请同学们开始实验

